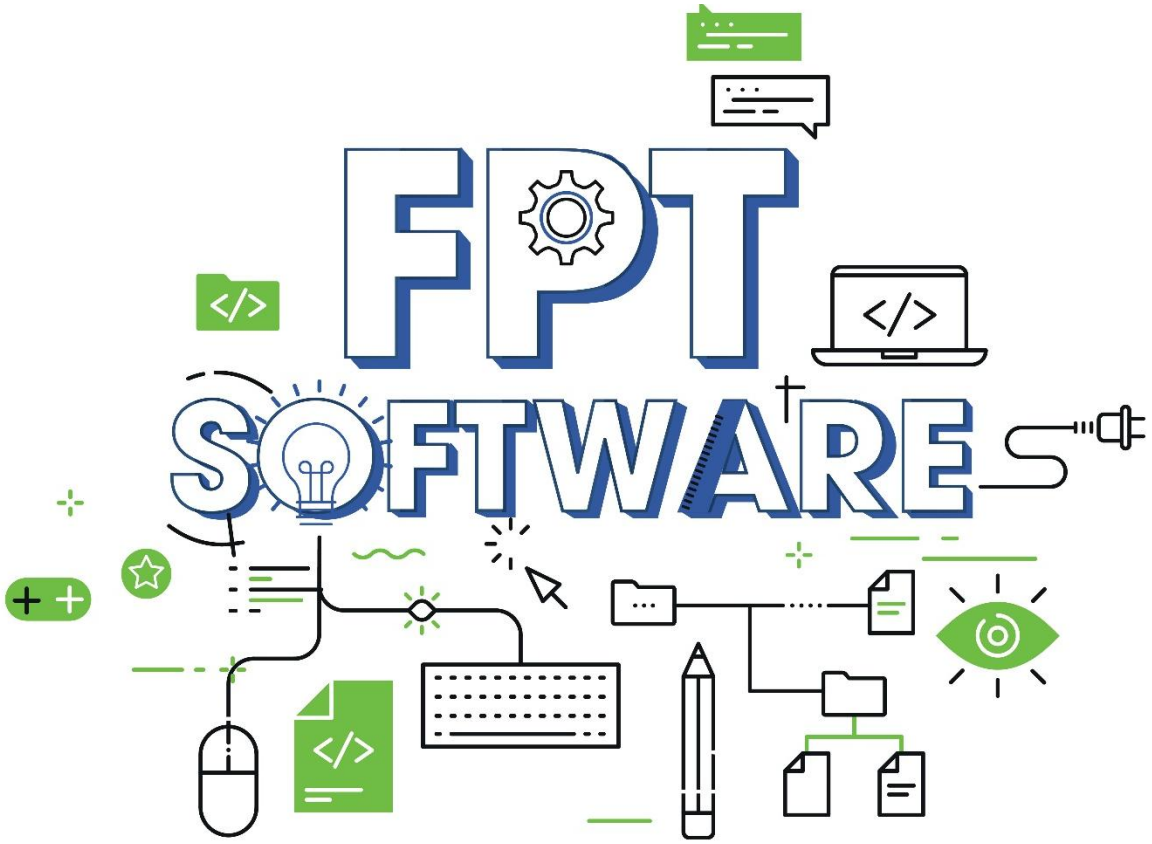
Basic Android

Android Services



Agenda

1. Introduction to Service
2. Role of Service
3. Types of Service in Android
4. Managing the lifecycle of a service
5. Interaction with services
6. Creating and Starting a Service
7. Alternatives to Service

1. Introduction to Service

A Service is an application component that **can perform long-running operations in the background**.

It does not provide a user interface. Once started, a service might **continue running for some time**, even after the user switches to another application.

Additionally, a component can bind to a service to interact with it and even perform inter-process communication (IPC).

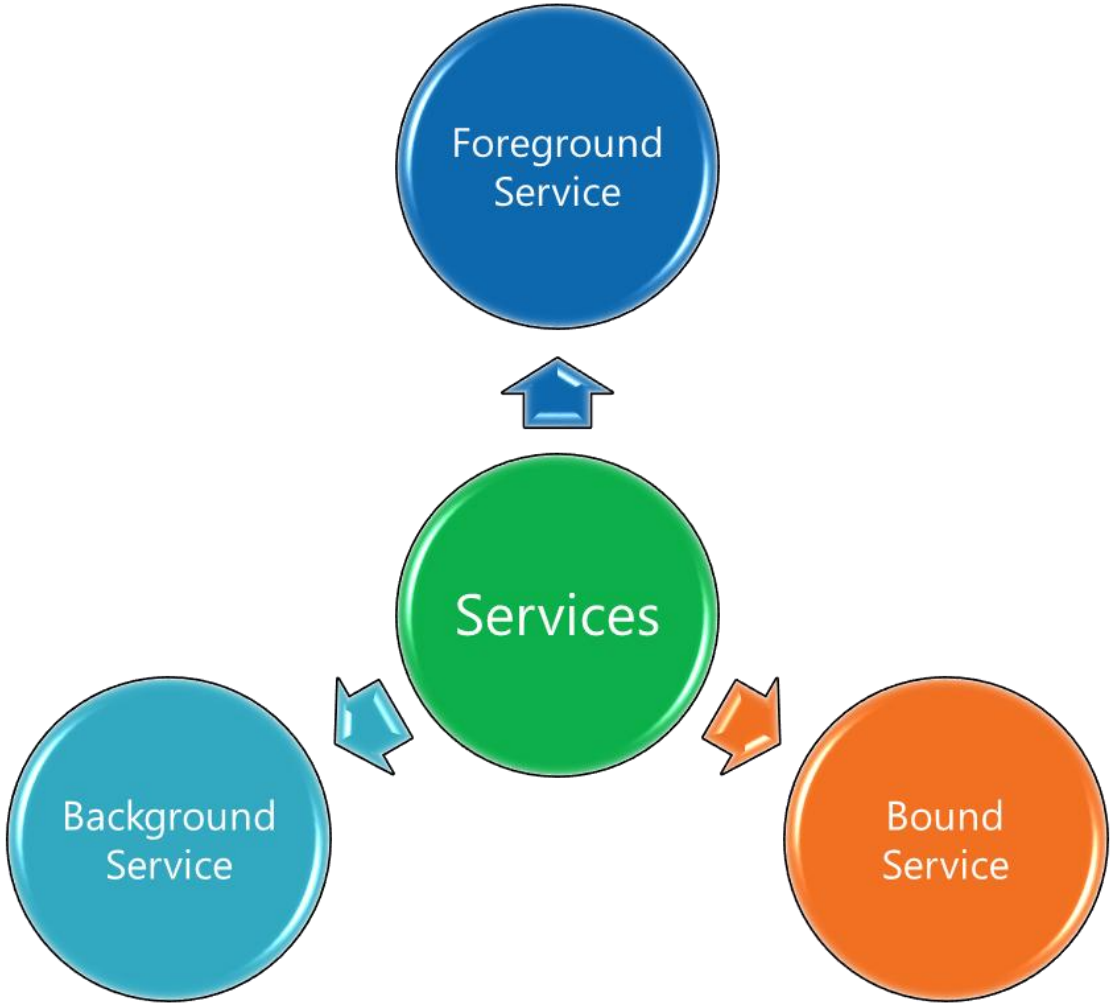
For example, a service can handle network transactions, play music, perform file I/O, or interact with a content provider, all from the background.

2. Role of Service

- Ensure that the application continues to operate even when the user switches to other apps or locks the screen.
- Execute long-running tasks without affecting user interaction with the application.
- Handle tasks that require access to system resources, such as downloading data or updating data from a server.
- Allow communication between different components within the application.

3. Types of Service in Android

In Android, there are three main types of services:



3. Types of Service in Android

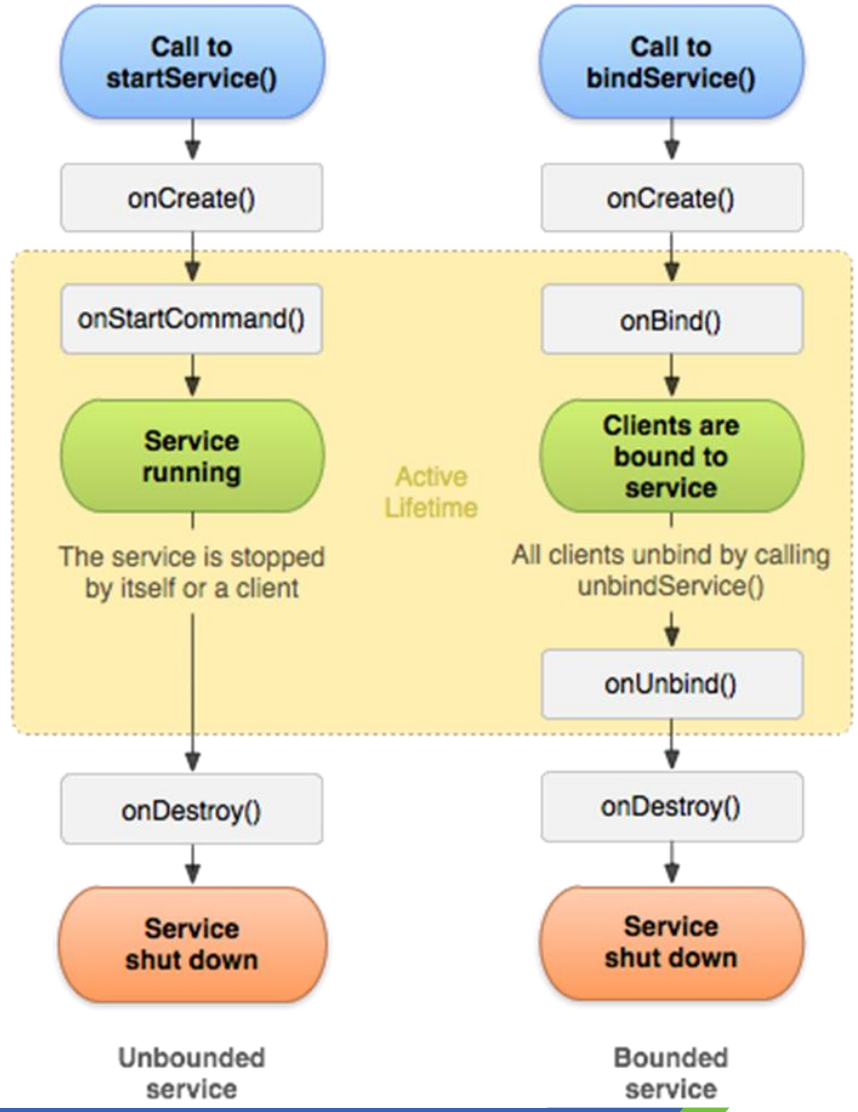
In Android, there are three main types of services:

- **Foreground Service**: Used when running a service with higher priority and requiring clear notification to the user about the service's operation. Foreground Service ensures that the service is less likely to be canceled by the system. This type of service is typically used for tasks that the user is actively aware of and may affect their experience with the app, such as music playback or a navigation service.
- **Background Service**: Used when no explicit notification to the user is required. However, starting from Android 10 (API level 29), Background Services that are not tied to the user interface may be terminated by the system after a certain period to save battery. This particularly applies to Background Services running below API level 26, as Android 8.0 (API level 26) introduced restrictions for Background Services running in the background. Background Services are typically used for tasks that are not immediately noticeable by the user, such as data synchronization or periodic updates.
- **Bound Service**: A Bound Service is a service that allows other components (typically activities) to bind to it and interact with it. The components can send requests to the service, get results from the service, or even perform interprocess communication (IPC). Bound Services are used for client-server interactions and are suitable when you need to perform tasks based on client requests.

4. Managing the lifecycle of a service

- **Foreground Service**: Use startForeground() to bring the service to the foreground state and display a notification to the user. This ensures that the service is less likely to be canceled by the system.
- **Background Service**: Avoid running continuous Background Services and stop them when they are no longer needed to save battery and system resources.

4. Managing the lifecycle of a service



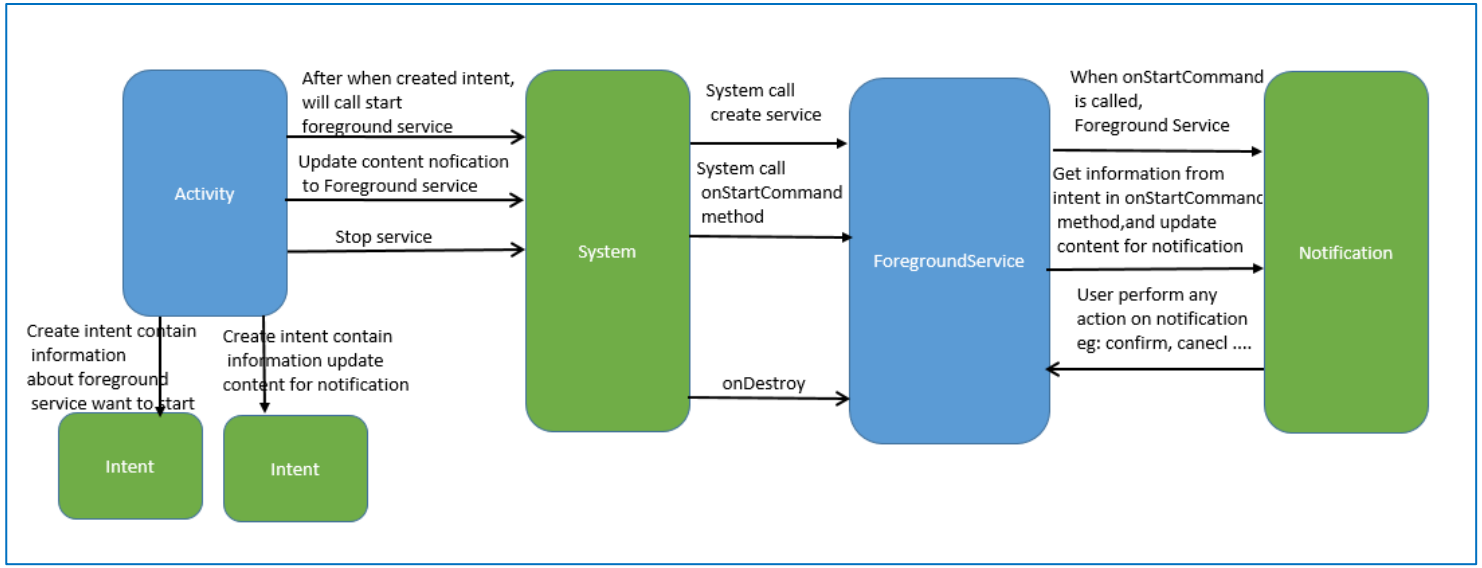
Methods	Description
onStartCommand()	The system invokes this method by calling startService() when another component (such as an activity) requests that the service be started. When this method executes, the service is started and can run in the background indefinitely. If you implement this, it is your responsibility to stop the service when its work is complete by calling stopSelf() or stopService(). If you only want to provide binding, you don't need to implement this method.
onBind()	This method is mandatory to implement in android service and is invoked whenever an application component calls the bindService() method in order to bind itself with a service. User-interface is also provided to communicate with the service effectively by returning an IBinder object. If the binding of service is not required then the method must return null.
onUnbind()	The Android system invokes this method when all the clients get disconnected from a particular service interface.
onRebind()	Once all clients are disconnected from the particular interface of service and there is a need to connect the service with new clients, the system calls this method.
onCreate()	The system invokes this method to perform one-time setup procedures when the service is initially created (before it calls either onStartCommand() or onBind()). If the service is already running, this method is not called.
onDestroy()	The system invokes this method when the service is no longer used and is being destroyed. Your service should implement this to clean up any resources such as threads, registered listeners, or receivers. This is the last call that the service receives.

5. Interaction with services

1. Interacting with Foreground Service:

Foreground Service is used when running a service with higher priority and requiring clear notification to the user about the service's operation. To interact with a Foreground Service, the Activity can use the startForegroundService() method to start the service. After the service is started, it will run in the background, and the user will see a notification on the status bar informing them that the service is running.

The Activity can interact with the Foreground Service using Intents and send specific requests to the service, such as controlling music playback or navigation.



5. Interaction with services

2. Interacting with Background Service:

Background Service is used when no explicit notification to the user is required. However, starting from Android 10 (API level 29), Background Services that are not tied to the user interface may be terminated by the system after a certain period to save battery. Background Services running below API level 26 face similar restrictions, as Android 8.0 (API level 26) introduced limitations for Background Services running in the background.

To interact with a Background Service, the Activity can also use the startService() method to start the service. However, no notification will be displayed to the user in this case.

Just like the Foreground Service, the Activity can interact with the Background Service using Intents and send specific requests to the service.

5. Interaction with services

3. Interacting with Bound Service:

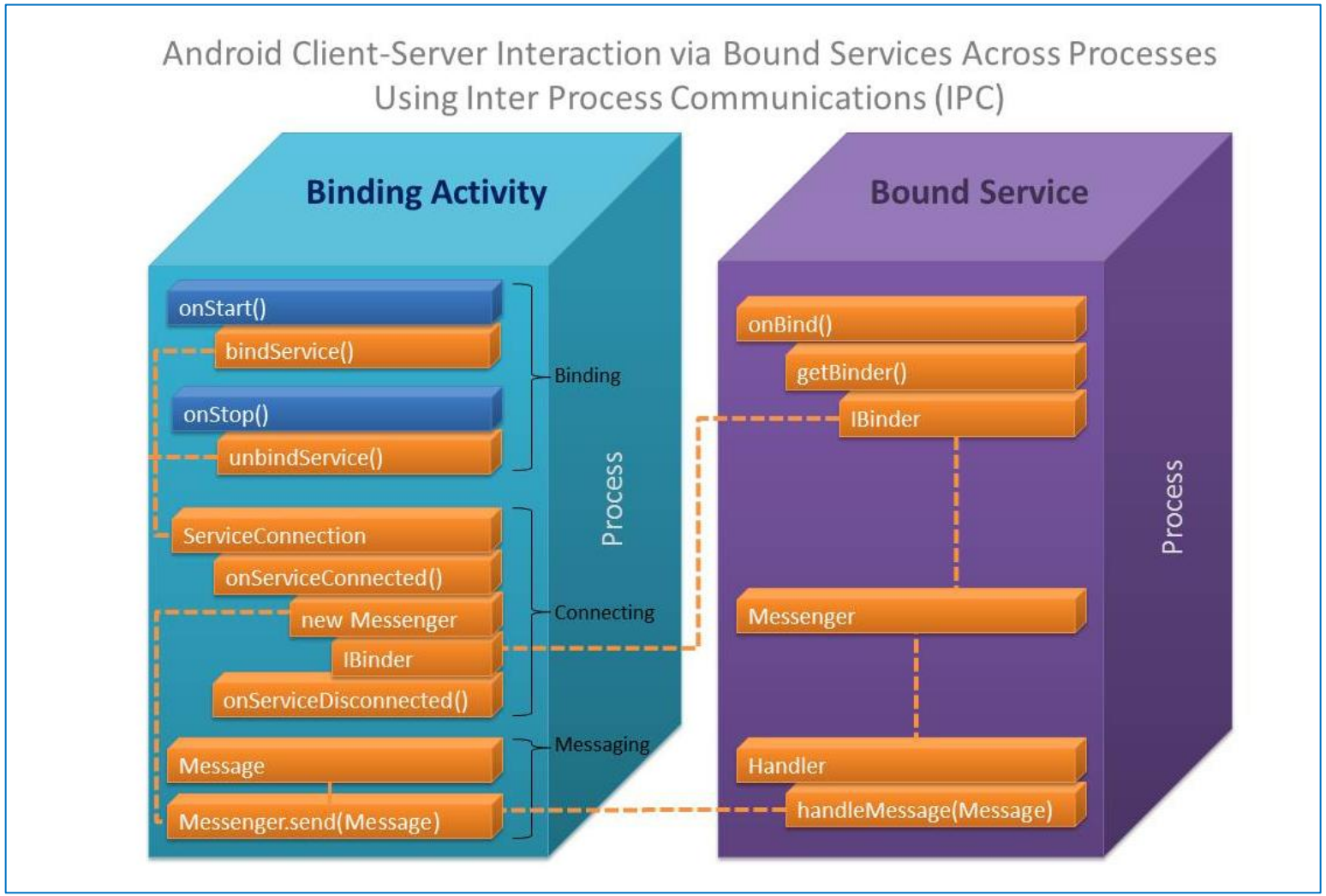
Bound Service is used when you want other components (typically activities) to bind to the service and interact with it. The Activity can use the bindService() method to bind with the service. When bound successfully, the Service returns a Binder object, allowing the Activity to call public methods defined in the Service.

Bound Service allows direct communication between the Activity and the Service through calling the public methods in the Service. The Activity can send specific requests to the Service and receive results back after the tasks are completed.

When no longer needed, the Activity should call the unbindService() method to release the connection with the Service.

5. Interaction with services

3. Interacting with Bound Service:



6. Creating and Starting a Service

6.1 Create **Foreground services**

Request the foreground service permission (target Android 9 (API level 28) or higher**)**

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android" ...>

    <uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>

    <application ...>
        ...
    </application>
</manifest>
```

Note: If an app that targets API level 28 or higher attempts to create a foreground service without requesting the **FOREGROUND_SERVICE** permission, the system throws a **SecurityException**.

6. Creating and Starting a Service

6.1 Create **Foreground services**

Create an class ForegroundService extends Service

```
public class ForegroundService extends Service {
    public static final String CHANNEL_ID = "ForegroundServiceChannel";
    @Override
    public void onCreate() {
        super.onCreate();
    }
    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        return START_NOT_STICKY;
    }
    @Override
    public void onDestroy() {
        super.onDestroy();
    }
    @Nullable
    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
```

Declaring a ForegroundService in the manifest

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    package="com.example.myapplication">

    <uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>

    <application>
        <service android:name=".ForegroundService"
            android:enabled="true"
            android:exported="true" />
    </application>

</manifest>
```

Notice that the onStartCommand() method must return an integer. The integer is a value that describes how the system should continue the service in the event that the system kills it. The return value from onStartCommand() must be one of the following constants:

START_NOT_STICKY

If the system kills the service after onStartCommand() returns, do not recreate the service unless there are pending intents to deliver. This is the safest option to avoid running your service when not necessary and when your application can simply restart any unfinished jobs.

START_STICKY

If the system kills the service after onStartCommand() returns, recreate the service and call onStartCommand(), but do not redeliver the last intent. Instead, the system calls onStartCommand() with a null intent unless there are pending intents to start the service. In that case, those intents are delivered. This is suitable for media players (or similar services) that are not executing commands but are running indefinitely and waiting for a job.

START_REDELIVER_INTENT

If the system kills the service after onStartCommand() returns, recreate the service and call onStartCommand() with the last intent that was delivered to the service. Any pending intents are delivered in turn. This is suitable for services that are actively performing a job that should be immediately resumed, such as downloading a file.

6. Creating and Starting a Service

6.1 Create **Foreground services**

Create an Notification for Service

In method onStartCommand we will is create an notification

```
@Override
public int onStartCommand(Intent intent, int flags, int startId) {
    String input = intent.getStringExtra( name: "inputExtra");
    createNotificationChannel();
    Intent notificationIntent = new Intent( packageContext: this, MainActivity.class);
    PendingIntent pendingIntent = PendingIntent.getActivity( context: this,
            requestCode: 0, notificationIntent, flags: 0);
    Notification notification = new NotificationCompat.Builder( context: this, CHANNEL_ID)
            .setContentTitle("Foreground Service")
            .setContentText(input)
            .setSmallIcon(R.drawable.icon_home)
            .setContentIntent(pendingIntent)
            .build();
    startForeground( id: 1, notification);
    //do heavy work on a background thread
    //stopSelf();
    return START_NOT_STICKY;
}
private void createNotificationChannel() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        NotificationChannel serviceChannel = new NotificationChannel(
                CHANNEL_ID,
                name: "Foreground Service Channel",
                NotificationManager.IMPORTANCE_DEFAULT
        );
        NotificationManager manager = getSystemService(NotificationManager.class);
        manager.createNotificationChannel(serviceChannel);
    }
}
```

6. Creating and Starting a Service

6.1 Create Foreground services

Start, Stop service by intent

Start

```
Intent serviceIntent = new Intent( packageContext: this, ForegroundService.class);
serviceIntent.putExtra( name: "inputExtra",  value: "Foreground Service Example in Android");
startForegroundService(serviceIntent);
```

Stop

```
Intent serviceIntent = new Intent( packageContext: this, ForegroundService.class);
stopService(serviceIntent);
```

6. Creating and Starting a Service

6.2 Create **Bound services**

Create an class BoundService extends Service
And create a internal class MyBinder extend Binder

```
public class BoundService extends Service {
    private static String LOG_TAG = "BoundService";
    private IBinder mBinder = new MyBinder();
    @Override
    public void onCreate() {
        super.onCreate();
        Log.v(LOG_TAG, msg: "in onCreate");
    }
    @Override
    public IBinder onBind(Intent intent) {
        Log.v(LOG_TAG, msg: "in onBind");
        return mBinder;
    }
    @Override
    public void onRebind(Intent intent) {
        Log.v(LOG_TAG, msg: "in onRebind");
        super.onRebind(intent);
    }
    @Override
    public boolean onUnbind(Intent intent) {
        Log.v(LOG_TAG, msg: "in onUnbind");
        return true;
    }
    @Override
    public void onDestroy() {
        super.onDestroy();
        Log.v(LOG_TAG, msg: "in onDestroy");
    }

    public class MyBinder extends Binder {
        BoundService getService() {
            return BoundService.this;
        }
    }
```

6. Creating and Starting a Service

6.2 Create **Bound services**

Declaring a BoundService in the manifest

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    package="com.example.myapplication">

    <application>
        <service android:name=".BoundService"
            android:enabled="true"
            android:exported="true" />
    </application>

</manifest>
```

6. Creating and Starting a Service

6.2 Create **Bound services**

In Activity get binder of boundService by ServiceConnection

```
public class MainActivity extends AppCompatActivity {

    private static final String TAG = "MainActivity";
    BoundService mService;
    private Button mBtnGetCurrentTime;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        mBtnGetCurrentTime = findViewById(R.id.btn_get_current_time);
        mBtnGetCurrentTime.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                if (mService != null) {
                    String currentTime = mService.getCurrentTime();
                    Toast.makeText(getApplicationContext(),currentTime,Toast.LENGTH_SHORT).show();
                }
            }
        });
    }

    @Override
    protected void onStart() {
        super.onStart();
        // Bind to LocalService.
        Intent intent = new Intent( packageContext: this, BoundService.class);
        bindService(intent, connection, Context.BIND_AUTO_CREATE);
    }
```

```
/** Defines callbacks for service binding, passed to bindService(). */
private ServiceConnection connection = new ServiceConnection() {

    @Override
    public void onServiceConnected(ComponentName className,
                                   IBinder service) {
        // We've bound to LocalService, cast the IBinder and get LocalService instance.
        BoundService.MyBinder binder = (BoundService.MyBinder) service;
        mService = binder.getService();
    }

    @Override
    public void onServiceDisconnected(ComponentName arg0) {
        mService = null;
    }
};

@Override
protected void onStop() {
    super.onStop();
    unbindService(connection);
}
```

6. Creating and Starting a Service

6.3 Create **Background services**

Create an class BackgroundService extends Service

```
public class BackgroundService extends Service {

    private MediaPlayer mPlayer;

    @Override
    public void onCreate() {
        super.onCreate();
    }

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
       //TODO implement logic
        return START_STICKY;
    }

    @Override
    public void onDestroy() {
        super.onDestroy();
    }

    @Nullable
    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
```

6. Creating and Starting a Service

6.3 Create **Background services**

Declaring a BoundService in the manifest

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    package="com.example.myapplication">

    <application>
        <service android:name=".BackgroundService" />
    </application>

</manifest>
```

6. Creating and Starting a Service

6.3 Create **Background services**

Implement logic play music in onStartCommand

```java
@Override
public int onStartCommand(Intent intent, int flags, int startId) {
    //getting systems default ringtone
    mPlayer = MediaPlayer.create(getApplicationContext(), Settings.System.DEFAULT_RINGTONE_URI);
    //setting loop play to true, this will make the ringtone continuously playing
    mPlayer.setLooping(true);
    //staring the player
    if(mPlayer.isPlaying()){
        mPlayer.stop();
    }else {
        mPlayer.start();
    }
    //Some options for service, start sticky means service will be explicitly started and stopped
    return START_STICKY;
}
```

6. Creating and Starting a Service

6.3 Create **Background services**

Start/Stop service in activity

MainActivity.java

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    btnStartService = findViewById(R.id.btn_start_service);
    btnStartService.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            startService();
        }
    });

    btnStopService = findViewById(R.id.btn_stop_service);
    btnStopService.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            stopService();
        }
    });
}

private void startService() {
    startService(new Intent(getApplicationContext(), BackgroundService.class));
}

private void stopService() {
    stopService(new Intent(getApplicationContext(),BackgroundService.class));
}
```

7. Alternatives to Service

When using Service in an Android application, there are some important considerations to ensure performance and a good user experience:

1. For large, heavy tasks, consider using other technologies such as WorkManager or JobScheduler:

Service in Android usually runs on the main UI thread of the application. If the application performs heavy tasks and runs them on the UI thread, it can cause UI lag, reduce user experience, and responsiveness of the application.

For large, heavy tasks that may take a significant amount of time, consider using other technologies such as WorkManager or JobScheduler. These technologies allow scheduling the execution of heavy tasks in a more optimized environment, including optimizing battery usage, system resources, and managing network connections. WorkManager and JobScheduler provide mechanisms to perform tasks periodically, asynchronously, and adaptively based on conditions.

7. Alternatives to Service

2. Avoid blocking the user interface (UI) thread by performing long-running tasks in a separate thread:

When performing long-running tasks in a Service, avoid executing them directly on the main UI thread. If long-running tasks are executed on the main thread, the application may become unresponsive, experience slow responsiveness, or receive low ratings on app stores.

Instead, perform long-running tasks in a separate thread. This can be achieved by using Thread or AsyncTask, or other asynchronous mechanisms such as Executor or Coroutine. By executing long-running tasks in a separate thread, we ensure that the task execution does not affect the UI thread, keeping the application responsive and snappy.

In summary, when using Service in an Android application, consider using other technologies such as WorkManager or JobScheduler for heavy tasks. Additionally, ensure that long-running tasks are executed in a separate thread to avoid blocking the UI thread and to improve the performance of the application.

THANK YOU!